📘 MANUAL TÉCNICO: INTEGRAÇÃO MVC (NODE.JS + HTML)

Este manual descreve a implementação profissional da arquitetura **MVC** para servir páginas estáticas e consumir dados de um banco de dados MySQL via API.

---

1. ARQUITETURA DE PASTAS (DIRETÓRIOS)

A organização deve separar o que é visível ao cliente (Front-end) do que é processado no servidor (Back-end).

- **`public/views/`**: Armazena o arquivo `index.html`. Esta pasta é considerada "estática".
- **`src/`**: Contém toda a inteligência do sistema.
  - **`Controller/`**: Lógica de decisão (recebe o pedido e envia a resposta).
  - **`Models/`**: Regras de negócio e acesso ao Banco de Dados.
  - **`Routers/`**: Definição das rotas (URLs) do sistema.
- **`server.js`**: Arquivo raiz que inicia o servidor Node.js.

---

2. CONFIGURAÇÃO DO SERVIDOR (`src/app.js`)

O arquivo `app.js` deve configurar o acesso aos arquivos estáticos para que o navegador consiga ler o HTML.

JavaScript
```
const express = require('express');
const path = require('path');
const dadosRoutes = require('./Routers/dadosRoutes');
const app = express();

// PASSO A PASSO:
// 1. Ensinamos o servidor a ler JSON.
app.use(express.json());
```

```
// 2. Definimos a pasta 'public' como estática para que o HTML
seja acessível.
app.use(express.static(path.join(__dirname, '../public')));

// 3. Conectamos as rotas.
app.use('/', dadosRoutes);

module.exports = app;
```

---

3. DEFINIÇÃO DE ROTAS (`src/Routers/dadosRoutes.js`)

Neste arquivo, separamos o que é a **página** do que são os **dados**.

JavaScript
```
const express = require('express');
const router = express.Router();
const path = require('path');
const dadosController =
require('../Controller/dadosController');

// ROTA DA PÁGINA: Entrega o arquivo visual.
router.get('/teste', (req, res) => {
    // Usamos '..' duas vezes para sair de 'Routers' e 'src'.
    res.sendFile(path.join(__dirname, '..', '..', 'public',
'views', 'index.html'));
});

// ROTA DE API: Entrega os dados do banco em formato JSON.
router.get('/listar-dados', dadosController.lista);

module.exports = router;
```

---

4. O FRONT-END (`public/views/index.html`)

O HTML utiliza a técnica de **Fetch API** para buscar os dados sem precisar atualizar a página (comportamento de SPA - Single Page Application).

HTML
```
<script>
    async function CarregarDados() {
        try {
            // 1. Fazemos o pedido para a rota de API.
            const resposta = await
fetch('http://localhost:3000/listar-dados');

            // 2. Convertemos a resposta para um objeto
JavaScript.
```

```
            const dados = await resposta.json();

            // 3. Selecionamos o elemento da lista no HTML.
            const lista = document.getElementById('lista-
dados');

            // 4. Percorremos os dados e criamos os itens
(<li>).
            dados.forEach(item => {
                const li = document.createElement('li');
                li.textContent = item.nome; // 'nome' deve ser a
coluna do seu banco.
                lista.appendChild(li);
            });
        } catch (error) {
            console.error('Erro na integração:', error);
        }
    }
    CarregarDados();
</script>
```

---

5. RESOLUÇÃO DE ERROS COMUNS (TROUBLESHOOTING)

Erro	Causa Provável	Solução Profissional
`MODULE_NOT_FOUND`	Erro de digitação no nome da pasta no `require`.	Verifique se a pasta é `Controller` (singular) ou `Controllers` (plural).
`ENOENT`	Caminho do arquivo (`path`) está incorreto.	Verifique a quantidade de pontos (`..`) para subir as pastas.
`ERR_CONNECTION_REFUSED`	O servidor não está ligado ou porta está errada.	Execute `node server.js` e verifique a porta no arquivo `.env`.